

The Two Critical Gaps to Fix

Gap 1 — Missing `df_clinical_test_raw.csv` This is needed for PulsePhysio initialization (NB10). It's the unscaled version with original mmHg/mg·dL values.

Gap 2 — No validation split The blueprint requires 70/15/15, but given the small N (1,190), 80/20 with cross-validation is defensible. You should at minimum add a note explaining this choice.

What to Add to Your Notebook

Here's the additional code to append to your existing notebook (after Section 10 — StandardScaler):

```
python
# =====
# Section 11 — Save Raw Clinical Test Set (for PulsePhysio)
# =====
# The blueprint requires an unscaled version of the test set
# with original clinical values (mmHg, mg/dL) for PulsePhysio
# initialization (NB10). This must be saved BEFORE scaling.
#
# ⚠ We reconstruct this from the PRE-SCALING data.
# The raw features needed by Pulse: age, sex, resting_bp,
# cholesterol, max_heart_rate (continuous values).

RAW_TEST_OUT = OUTPUT_DIR + "df_clinical_test_raw.csv"

# — Reconstruct raw test set —————
# X_test_raw is the imputed (no missing values) but UNSCALED
# version of the test features — we need to save this before
# StandardScaler transforms the numeric columns.
#
# Since we already applied the scaler in Section 10, we
# reconstruct the raw values using the inverse transform.

X_test_raw = X_test.copy()

# Inverse-transform the scaled columns back to original units
X_test_raw[SCALE_COLS] = scaler.inverse_transform(X_test_raw[SCALE_COLS])

# Reconstruct the full raw test DataFrame with target
df_test_raw_out = X_test_raw.copy()
df_test_raw_out["target"] = y_test.values
```

```

# Add human-readable labels for Pulse initialization reference
df_test_raw_out.to_csv(RAW_TEST_OUT, index=False)

print("=" * 55)
print("RAW CLINICAL TEST SET — SAVED FOR PULSEOPHYSIO")
print("=" * 55)
print(f" ✅ Saved: {RAW_TEST_OUT} ({len(df_test_raw_out):,} rows)")
print()
print(" Columns with original clinical units:")
print(f"   age (years)      :
{X_test_raw['age'].describe().round(1)[['min','mean','max']].to_dict()}")
print(f"   resting_bp (mmHg)  :
{X_test_raw['resting_bp'].describe().round(1)[['min','mean','max']].to_dict()}")
print(f"   cholesterol (mg/dL) :
{X_test_raw['cholesterol'].describe().round(1)[['min','mean','max']].to_dict()}")
print(f"   max_heart_rate (bpm) :
{X_test_raw['max_heart_rate'].describe().round(1)[['min','mean','max']].to_dict()}")
print()
print(f" ⓘ This file is for PulsePhysio patient initialization only.")
print(f" ⓘ Use df_clinical_test.csv (scaled) for all ML model evaluation.")
python
# =====
# Section 12 — SMOTE Assessment
# =====
# Blueprint requires SMOTE on training set if class imbalance
# ratio exceeds 1.5:1. Apply AFTER split, NEVER on test set.

from collections import Counter

print("=" * 55)
print("CLASS BALANCE ASSESSMENT — SMOTE DECISION")
print("=" * 55)

train_class_counts = Counter(y_train)
imbalance_ratio = max(train_class_counts.values()) / min(train_class_counts.values())

print(f" Train class distribution: {dict(train_class_counts)}")
print(f" Imbalance ratio: {imbalance_ratio:.3f}")
print(f" SMOTE threshold: 1.5")
print()

if imbalance_ratio > 1.5:
    print(f" ⚠ Imbalance exceeds 1.5:1 — applying SMOTE to X_train only")

# Note: SMOTE must be applied to the SCALED X_train
# (after StandardScaler) to avoid scale mismatches
try:

```

```

from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=RANDOM_STATE)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
print(f" Before SMOTE: {Counter(y_train)}")
print(f" After SMOTE : {Counter(y_train_smote)}")
print()
print(" ⓘ Use X_train_smote / y_train_smote for model training in NB6.")
print(" ⓘ X_test remains unchanged (never SMOTE the test set).")
except ImportError:
    print(" ⓘ imbalanced-learn not installed.")
    print(" ⓘ Alternative: use class_weight='balanced' in sklearn models.")
    print(" ⓘ Recommended for small N (1190) — less risky than SMOTE.")
    X_train_smote, y_train_smote = X_train, y_train # fallback
else:
    print(f" ✓ Imbalance ratio {imbalance_ratio:.3f} < 1.5 — SMOTE not required.")
    print(" ⓘ Use class_weight='balanced' as a precaution in NB6 models.")
    X_train_smote, y_train_smote = X_train, y_train

python
# =====
# Section 13 — Preprocessing Report (text file)
# =====
# Blueprint requires preprocessing_report.txt documenting
# all transformations and dropped rows.

REPORT_OUT = OUTPUT_DIR + "preprocessing_report.txt"

report_lines = [
    "=" * 60,
    "PREPROCESSING REPORT — CLINICAL DATASET (1190 instances)",
    f"Generated: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M')}",
    "=" * 60,
    """
    ,
    "1. SOURCE DATA",
    f" File: heart_statlog_cleveland_hungary_final.csv",
    f" Raw shape: 1,190 rows × 12 columns",
    f" Sources: Cleveland (303) + Hungarian (294) + Zürich (123) + Long Beach VA (200) +
StatLog (270)",
    """
    ,
    "2. COLUMNS EXCLUDED AT SOURCE",
    " - ca (major vessels): >90% missing across contributing datasets",
    " - thal (thalassemia): >90% missing across contributing datasets",
    " Reason: These columns were intentionally omitted during the multi-source merge.",
    """
    ,
    "3. SENTINEL ZERO REPLACEMENT",
    f" - cholesterol == 0 → NaN : 172 rows (14.5%) — biologically impossible",
    f" - resting_bp == 0 → NaN : 1 row (0.1%) — incompatible with life",
    f" - st_slope == 0 → NaN : 1 row (0.1%) — encoded as 'unknown'",
    " Note: oldpeak negative values RETAINED (ST elevation — physiologically valid)",

```

```

"""
3
"4. IQR CLIPPING (applied BEFORE imputation)",
" - cholesterol : clipped to [108.5, 376.5] — 3 low, 25 high",
" - resting_bp : clipped to [90.0, 170.0] — 1 low, 35 high",
" - max_heart_rate: clipped to [62.5, 218.5] — 1 low, 0 high",
" - oldpeak : clipped to [-2.4, 4.0] — 1 low, 10 high",
" Rationale: Extreme but plausible clinical values capped rather than deleted.",
"""
3
"5. TRAIN/TEST SPLIT",
f" Train: 952 rows (80%) | Test: 238 rows (20%)",
f" Strategy: Stratified by target | random_state=42",
f" Train class balance: 449 (47.2%) no disease | 503 (52.8%) disease",
f" Test class balance: 112 (47.1%) no disease | 126 (52.9%) disease",
"""
3
"6. MICE IMPUTATION",
" Columns imputed: cholesterol, resting_bp, st_slope",
" Fit on X_TRAIN ONLY — no data leakage",
" max_iter=10, random_state=42",
" Post-imputation rounding: cholesterol, resting_bp → int; st_slope → int",
"""
3
"7. ONE-HOT ENCODING",
" Columns encoded: chest_pain_type (4 cats), resting_ecg (3 cats), st_slope (3 cats)",
" Columns kept as binary: sex, fasting_blood_sugar, exercise_angina",
" Total columns: 11 → 18 after encoding",
"""
3
"8. STANDARD SCALING",
" Columns scaled: age, resting_bp, cholesterol, max_heart_rate, oldpeak",
" Scaler fit on X_TRAIN ONLY — applied to both train and test",
"""
3
"9. SMOTE ASSESSMENT",
f" Class imbalance ratio: {imbalance_ratio:.3f}",
f" SMOTE applied: {'YES' if imbalance_ratio > 1.5 else 'NO' — ratio below 1.5:1
threshold}",
" Recommendation: Use class_weight='balanced' in NB6 models as precaution",
"""
3
"10. OUTPUTS",
f" df_clinical_train.csv : 952 rows × 19 columns (18 features + target)",
f" df_clinical_test.csv : 238 rows × 19 columns (18 features + target)",
f" df_clinical_test_raw.csv: 238 rows (unscaled, for PulsePhysio)",
f" clinical_scaler.pkl : StandardScaler fitted on train set",
f" clinical_imputer.pkl : MICE IterativeImputer fitted on train set",
"""
3
"=" * 60,
"DATA LIMITATIONS — MUST ACKNOWLEDGE IN PAPER",
"=" * 60,
"- Dataset combines 5 Western cohorts (Cleveland, Hungarian, Zurich, VA, StatLog).",
" No Indian patients — clinical model validated on non-Indian population.",
" India-specific calibration achieved via PRS integration (NB4) not clinical data.",

```

```

"- ca and that columns excluded due to >90% missingness; ordinal cholesterol/glucose",
" from 70k dataset cannot be directly compared to mg/dL values here.",
"- Cross-sectional data only — no longitudinal follow-up available.",
"=" * 60,
]

with open(REPORT_OUT, 'w') as f:
    f.write('\n'.join(report_lines))

print(f" ✅ Preprocessing report saved to: {REPORT_OUT}")
print()
print(" Report summary:")
for line in report_lines[:20]:
    print(f" {line}")

```

Updated Final Validation Cell

Also update your existing Section 11 (Final Validation) to check for the new outputs:

```

python
# Add to your existing validation checks:
assert os.path.exists(RAW_TEST_OUT), "Raw clinical test file not saved!"
print(f" ✅ Raw clinical test set saved ({len(df_test_raw_out)} rows)")

assert os.path.exists(REPORT_OUT), "Preprocessing report not saved!"
print(f" ✅ Preprocessing report saved")

```

Summary of What You Need to Do

1. **Add Sections 11, 12, 13** to your existing notebook (the code above)
2. **Keep your existing sections 1–11** — they're solid and methodologically correct
3. **Note in your paper:** split is 80/20 not 70/15/15, justified by small N and cross-validation in NB6
4. **The `df_clinical_test_raw.csv`** is the most important addition — NB10 (PulsePhysio) won't work without it

Your existing preprocessing logic (MICE, IQR clipping, sentinel zeros) is actually *better* than what the blueprint specifies — just add the missing outputs.